

MiniHTTPd

Los servicios web actuales están disponibles las 24 horas durante todo el año. Para mantener esta disponibilidad, su arquitectura debe implementar concurrencia y uso eficiente de recursos en el servidor, además, el tráfico de estas plataformas compromete actividades económicas, servicios digitales y experiencias de usuario que dependen de una respuesta continua y estable.

Desarrollar un mini servidor HTTPd en C para Linux permite comprender el funcionamiento a bajo nivel de un servidor real y la comunicación mediante sockets TCP. Por ello, el proyecto MiniHTTPd tiene como objetivo implementar, de manera básica, herramientas de concurrencia y gestión de conexiones en un entorno Linux.

El proyecto se realizó en WSL Ubuntu, con lenguaje C. El servidor utiliza el protocolo HTTP/1.1 y sirve archivos estáticos HTML, imágenes PNG y JPG y archivos Javascripts, mediante detección de tipos MIME. El servidor únicamente soporta el método GET, por eso las solicitudes `curl -I` devuelven el código de estado 405 Method Not Allowed, porque usa el método HEAD. En cambio, `curl -i` permite verificar encabezados HTTP y tipos MIME del archivo solicitado. Por esta razón, la etapa de parsing y validación de solicitudes GET fue la más extensa del desarrollo. Adicionalmente, el servidor implementa medidas básicas de seguridad y manejo de errores mediante códigos de estado para validar solicitudes inválidas y recursos inexistentes.

La concurrencia se implementó mediante `epoll` y sockets no bloqueantes. Un socket principal acepta conexiones entrantes y delega la gestión de múltiples clientes a un hilo. Este modelo event driven responde solo a las conexiones activas y optimiza el uso de recursos del sistema para atender miles de peticiones concurrentes. La concurrencia fue evaluada con `ApacheBench`, en la prueba el servidor procesó con éxito 10000 peticiones HTTP con una carga concurrente de 500 clientes. En el desarrollo se generaron archivos de metadatos `Zone.Identifier` asociados a Windows. Los archivos se crean automáticamente al copiar el directorio del proyecto desde Windows a WSL Ubuntu pero no interfieren con el proyecto y pueden ser eliminados con la terminal.

Al finalizar el proyecto, fue posible entender por qué los servicios web fallan, tienen retrasos o no están disponibles. Solamente hace falta que un socket se bloquee, que un parser HTTP falle o que exista una vulnerabilidad en el manejo de solicitudes para afectar el funcionamiento entero del servidor. Cada componente del sistema debe estar implementado correctamente, desde los sockets TCP hasta mecanismos de concurrencia `epoll`. Estas tecnologías no funcionan de manera aislada, sino como partes dependientes entre sí de un sistema que requiere estabilidad, eficiencia y manejo de recursos para responder correctamente bajo alta demanda.

Repositorio: <https://github.com/DavidPilataxi/MiniHTTPd.git>